

Lab-8.1

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task 1: Password Strength Validator Task

Description

Create a function `is_strong_password(password)` that returns `True` if the password meets these rules:

- At least 8 characters
- Contains at least one uppercase letter, one lowercase letter, one digit, and one special character (non-alphanumeric)
- No spaces

Prompt for AI :-

“Generate at least 3 assert test cases for a function `is_strong_password(password)` that checks password strength according to the rules: minimum length 8, must contain uppercase, lowercase, digit, special character, and no spaces. Include edge cases.”

Code (test cases + implementation)

Python:-

```
1 # Test cases (AI-generated)
2 def test_password_validator():
3     assert is_strong_password("Abcd@123") == True # strong
4     assert is_strong_password("abcd123") == False # missing uppercase & special
5     assert is_strong_password("ABCD@1234") == True # strong
6     assert is_strong_password("Abcd 123") == False # contains space
7     assert is_strong_password("Abc@12") == False # too short
8
9 # Implementation (passes all tests)
10 def is_strong_password(password: str) -> bool:
11     if len(password) < 8 or ' ' in password:
12         return False
13     if not any(c.isupper() for c in password):
14         return False
15     if not any(c.islower() for c in password):
16         return False
17     if not any(c.isdigit() for c in password):
18         return False
19     if not any(not c.isalnum() for c in password):
20         return False
21     return True
22
23 # Run tests
24 test_password_validator()
25 print("Task 1 passed")
```

Task 2: Number Classification

Task Description

Write a function `classify_number(n)` that returns "Positive", "Negative", or "Zero".

- Use loops in the test (e.g., iterate over test cases)
- Handle invalid inputs like strings and None

Prompt for AI (to generate test cases)

“Generate at least 3 assert test cases for a function `classify_number(n)`. The function should return 'Positive', 'Negative', or 'Zero'. Use a loop to test multiple values, including boundary conditions (-1, 0, 1). Also include tests for invalid inputs like strings and None, expecting a `TypeError`.”

Code (test cases + implementation)

Python:-

```

1 def test_number_classifier():
2     test_data = [
3         (10, "Positive"),
4         (-5, "Negative"),
5         (0, "Zero"),
6         (1, "Positive"),
7         (-1, "Negative"),
8     ]
9     for num, expected in test_data:
10        assert classify_number(num) == expected
11    try:
12        classify_number("hello")
13        assert False, "Should have raised TypeError"
14    except TypeError:
15        pass
16    try:
17        classify_number(None)
18        assert False, "Should have raised TypeError"
19    except TypeError:
20        pass
21    def classify_number(n):
22        if not isinstance(n, (int, float)):
23            raise TypeError("Input must be a number")
24        if n > 0:
25            return "Positive"
26        elif n < 0:
27            return "Negative"
28        else:
29            return "Zero"
30    test_number_classifier()

```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

Task 2 passed

Task 3: Anagram Checker

Task Description

Implement `is_anagram(str1, str2)` that returns `True` if the two strings are anagrams.

- Ignore case, spaces, and punctuation
- Handle edge cases like empty strings, identical words

Prompt for AI (to generate test cases)

"Generate at least 3 assert test cases for a function `is_anagram(str1, str2)`. The function should ignore case, spaces, and punctuation. Include edge cases such as empty strings and identical words."

Code (test cases + implementation)

Python:-

```

1 # Test cases (AI-generated)
2 def test_anagram():
3     assert is_anagram("listen", "silent") == True
4     assert is_anagram("hello", "world") == False
5     assert is_anagram("Dormitory", "Dirty Room") == True
6     assert is_anagram("", "") == True
7     assert is_anagram("a", "a") == True
8     assert is_anagram("A gentleman", "Elegant man") == True
9 # Implementation (passes all tests)
10 def is_anagram(str1: str, str2: str) -> bool:
11     def clean(s: str) -> str:
12         return "".join(ch.lower() for ch in s if ch.isalnum())
13     return sorted(clean(str1)) == sorted(clean(str2))
14 # Run tests
15 test_anagram()
16 print("Task 3 passed")

```

Shell ×

```
>>> %Run -c $EDITOR_CONTENT
```

```
Task 3 passed
```

Task 4: Inventory Class

Task Description

Create an Inventory class with methods:

- `add_item(name, quantity)`
- `remove_item(name, quantity)`
- `get_stock(name)`

Prompt for AI (to generate test cases)

“Generate at least 3 assert-based tests for an Inventory class that manages stock. The class should have methods add_item, remove_item, and get_stock. Include tests for adding, removing, and retrieving stock, as well as edge cases like removing more than available or negative quantities.” Code (test cases + implementation)

Python:-

```
1 # Test cases (AI-generated)
2 def test_inventory():
3     inv = Inventory()
4     inv.add_item("Pen", 10)
5     assert inv.get_stock("Pen") == 10
6     inv.remove_item("Pen", 5)
7     assert inv.get_stock("Pen") == 5
8     inv.add_item("Book", 3)
9     assert inv.get_stock("Book") == 3
10    # Edge cases
11    try:
12        inv.remove_item("Pen", 10) # not enough stock
13        assert False
14    except ValueError:
15        pass
16    try:
17        inv.add_item("Pen", -1) # negative quantity
18        assert False
19    except ValueError:
20        pass
21    # Implementation (passes all tests)
22    class Inventory:
23        def __init__(self):
24            self._stock = {}
25        def add_item(self, name: str, quantity: int) -> None:
26            if quantity < 0:
27                raise ValueError("Quantity cannot be negative")
28            self._stock[name] = self._stock.get(name, 0) + quantity
29        def remove_item(self, name: str, quantity: int) -> None:
30            if quantity < 0:
31                raise ValueError("Quantity cannot be negative")
32            if name not in self._stock or self._stock[name] < quantity:
33                raise ValueError(f"Not enough stock for '{name}'")
34            self._stock[name] -= quantity
35            if self._stock[name] == 0:
36                del self._stock[name]
37        def get_stock(self, name: str) -> int:
38            return self._stock.get(name, 0)
39    # Run tests
40    test_inventory()
41    print("Task 4 passed")
```

Shell x

```
>>> %Run -c $EDITOR_CONTENT
```

```
Task 4 passed
```

Task 5: Date Validation & Formatting

Task Description:-

Write `validate_and_format_date(date_str)` that:

- Validates the string in MM/DD/YYYY format
- Returns "Invalid Date" if invalid
- Otherwise returns the date as YYYY-MM-DD

Prompt for AI :-

"Generate at least 3 assert test cases for a function `validate_and_format_date(date_str)`. It should validate dates in MM/DD/YYYY format and convert valid dates to YYYY-MM-DD. Include tests for invalid dates (like Feb 30) and edge cases (leap years, wrong month/day)."

Code (test cases + implementation)

Python:-

```
1 # Test cases (AI-generated)
2 def test_date_validator():
3     assert validate_and_format_date("10/15/2023") == "2023-10-15"
4     assert validate_and_format_date("02/30/2023") == "Invalid Date"
5     assert validate_and_format_date("01/01/2024") == "2024-01-01"
6     assert validate_and_format_date("13/01/2024") == "Invalid Date"
7     assert validate_and_format_date("02/29/2024") == "2024-02-29" # leap year
8     assert validate_and_format_date("02/29/2023") == "Invalid Date" # non-leap
9 # Implementation (passes all tests)
10 import re
11 from datetime import datetime
12 def validate_and_format_date(date_str: str) -> str:
13     if not re.match(r'^\d{2}/\d{2}/\d{4}$', date_str):
14         return "Invalid Date"
15     try:
16         month, day, year = map(int, date_str.split('/'))
17         datetime(year, month, day) # raises ValueError if invalid date
18         return f"{year:04d}-{month:02d}-{day:02d}"
19     except ValueError:
20         return "Invalid Date"
21 # Run tests
22 test_date_validator()
23 print("Task 5 passed")
```

hell x

·> %Run -c \$EDITOR_CONTENT

Task 4 passed